Parking Sign Studio — Demo

A zero-backend, single-file web app for generating printable **A4-landscape parking signs**. Ships with **8 preset templates** for common venue types (hotel, clinic, office, restaurant, residence, event venue, loading bay, blank). Tap a template, the whole app reconfigures — branding, colours, sign language, contact defaults — live preview on the right, hit print.

Show Image

Show Image

Show Image

Show Image

What this is (and isn't)

This is a **demo / showcase** version of the generator. If you're deploying it for a single organisation, use the production `parking.html` instead — it's set up for one brand at a time, with cleaner CONFIG.

Use this demo when you want to:

- **Evaluate the tool** — click through the 8 templates and see whether it fits your venue
- **Show the tool to a client** — point them at the deployed demo, they pick the template closest to their business and see a realistic sign

- **Prototype a new template** — add your organisation to the `TEMPLATES` object, iterate quickly
- **Use it as-is for a small venue** — pick the template closest to your needs, edit the fields live, print. No rebranding required.

Features

- **8 preset templates** covering the most common venue types — switch between them with a single tap
- **Live A4 preview** at true 297 × 210 mm size
- **5 languages** for the sign title — English, Bahasa Malaysia, 中文, தமிழ், ਪੰਜਾਬੀ — with pick-a-primary / pick-a-secondary dropdowns
- **Inline SVG logos** — generated on the fly from template initials, so no external image files are needed. The whole app is literally one HTML file.
- **Colour & B&W modes** — B&W mode goes full mono on the sign *and* neutralises the sidebar yellow, for toner-cheap printing
- **Optional QR code** for the sign, with presets (Website / Sign details / Clear)
- **Optional contact QR** that auto-detects phone numbers (→ `tel:` for scan-to-call), emails (→ `mailto:`), or arbitrary text
- **Smart room-label** — the badge next to the guest name says "Room", "Suite", "Floor", "Table", "Unit", "Hall", "Bay" depending on the template
- **Signature strip** toggle — adds Authorised / Signature / Date lines at the bottom of the sign
- **Expired-warning ribbon** — diagonal red "EXPIRED" banner overlays the sign if the end date has already passed (you can still print it if you need to, you just can't miss it)
- **Past-date validation** — date inputs turn red with a warning if you pick a past date, but it's not blocked (backdated passes are a real use case)
- **Advanced settings toggle** — collapses away make/model, language picker, contact editor, signature toggle, and history for a minimal default view. State persists across sessions.
- **Collapsible sections** — every section fold-and-restores, preferences saved to `localStorage`
- **Sticky action bar** — Reset / Save PDF / Print always visible at the bottom of the sidebar
- **Recent signs history** — last 10 printed signs saved locally, one-click reload, tagged with the template they were made under
- **Accessibility** — real `<button>` elements everywhere, proper `<label for="">` pairings, `aria-pressed` and `aria-checked` on all toggles, `lang` attributes on each rendered sign title, `role="img"` on the preview canvas, visible focus rings
- **Two keyboard shortcuts** — `Ctrl/Cmd + Enter` to Print, `Ctrl/Cmd + S` to Save PDF

Templates included

Key	Display name	Brand	Title (EN)	Accent colour	Logo text
`hotel`	Hotel	Grand Harbour Hotel	RESERVED PARKING	Gold `#c9a227`	`GH`
`clinic`	Clinic	Wellspring Medical Centre	PATIENT PARKING	Teal `#0891b2`	✚
`office`	Office	Axis Tower	VISITOR PARKING	Blue `#2563eb`	`AX`
`restaurant`	Restaurant	Bamboo Garden	CUSTOMER PARKING	Red `#dc2626`	`BG`
`residence`	Residence	Marina Heights Residences	RESIDENT PARKING	Green `#059669`	`MH`
`event`	Event	Pavilion Convention Centre	EVENT PARKING	Purple `#7c3aed`	`PAV`
`loading`	Loading	Central Distribution Hub	LOADING BAY	Orange `#ea580c`	`CDH`
`blank`	Blank	Your Organisation	RESERVED PARKING	Yellow `#facc15`	`YO`

Each template also carries translations of its sign title into Bahasa Malaysia, Chinese (Simplified), Tamil, and Punjabi (Gurmukhi), plus a sensible contact role label ("Security", "Reception", "Front Desk", "Manager", "Management", "Event Security", "Dock Supervisor", "Contact"), a realistic default plate, a realistic default guest name, and a venue-appropriate room-badge label.

Quick start

1. Open `parking-demo.html` in any modern browser. Works locally from disk (no server needed) or from any static host.

2. The **Hotel** template loads by default — click any other template card to switch. Everything re-renders instantly.
3. Fill in the plate, guest name, dates.
4. Hit **Print** (or `Ctrl/Cmd + Enter`).

First-time users: make sure **Background graphics** is ticked in the browser's print dialog, otherwise the dark plate box won't print. This is the single most common reason signs come out wrong.

Adding a new template

Open `parking-demo.html`, find the `TEMPLATES` object near the top of the `<script>` block, and add an entry. Here's the anatomy of a template, with every field explained:

```js
```

```
const TEMPLATES = {
  // ...existing templates...

  school: {                                   // key used internally (must be unique)
    name: "School",                           // label shown on the template card
    brand: {
      name:            "Riverside Academy",   // shown in the footer of the sign + page titl
      enforcementText: "Staff and registered visitors only",  // small disclaimer line
      websiteUrl:      "https://example.edu"  // used by the "Website" QR preset
    },
    contact: {
      label: "School Office",                 // default contact label (editable live)
      value: "+60 3-1234 5678"                // default contact value
    },
    signTitles: {
      en: "STAFF PARKING",                    // English title (required — fallback)
      bm: "PARKIR KAKITANGAN",                // Bahasa Malaysia
      zh: "教职员工停车位",                      // Chinese (Simplified)
      ta: "ஊழியர் பார்க்கிங்",                   // Tamil
      pa: "ਸਟਾਫ਼ ਪਾਰਕਿੰਗ"                    // Punjabi (Gurmukhi)
    },
    colors: {
      accent: "#0369a1",                      // primary highlight on the sign
      dark:   "#0c4a6e"                       // near-black for frames and text
    },
    logo: {
      text:  "RA",                            // 1–3 chars, rendered in an SVG box
      color: "#0369a1"                        // background of the logo box
    },
    defaults: {
      plate:     "WQR 4501",                  // pre-filled example plate
      guestName: "TEACHING STAFF",            // pre-filled example name
      roomLabel: "Block"                      // what the badge next to the guest name says
    }
  }
};
```

Then re-open the file in the browser. A new **School** card appears at the end of the template grid. That's it.

Template card colour

The small coloured square on each template card uses `logo.color`. Pick something that

contrasts against the dark sidebar (`#0f172a`) — pure black / very dark colours blend in. Anything mid-tone or brighter works.

Keeping the grid tidy

The template grid is 4 columns wide. Adding a 9th template will wrap it to a third row. For best visual balance, either keep it to 8 (swap one out), or add up to 12 (a full three rows). Any number works, it just looks cleanest at multiples of 4.

Language translations

Default translations are provided for all 5 languages, but **these are approximations** — regional wording varies, and honestly some phrases are more commonly written in English even in non-English signage. If a native speaker flags something as awkward, update the relevant `signTitles` entry — no other code needs to change.

Recommended alternates if the defaults feel off:

English	Default Malay	Common alternates
RESERVED PARKING	PARKIR KHAS	TEMPAT LETAK KERETA RIZAB · PARKIR RIZAB
PATIENT PARKING	PARKIR PESAKIT	TEMPAT LETAK PESAKIT
VISITOR PARKING	PARKIR PELAWAT	TEMPAT LETAK PELAWAT
CUSTOMER PARKING	PARKIR PELANGGAN	TEMPAT LETAK PELANGGAN
RESIDENT PARKING	PARKIR PENGHUNI	TEMPAT LETAK PENGHUNI
LOADING BAY	TEMPAT PUNGGAH	TEMPAT MEMUAT

Fonts for Chinese, Tamil, and Punjabi are loaded via Google Fonts (Noto Sans SC / Tamil / Gurmukhi) and swap-displayed, so the title looks clean even in non-Latin scripts.

How the inline logos work

Each template doesn't ship with a `.png`. Instead, a tiny JavaScript function generates a logo as an **SVG data URI** on the fly:

```js
```

```
function makeLogoSVG(text, bgColor) {
  const svg = '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 260 120">' +
                '<rect width="260" height="120" rx="16" fill="' + bgColor + '"/>' +
                '<text x="130" y="84" font-family="Arial,Helvetica,sans-serif" ' +
                  'font-size="62" font-weight="900" fill="#fff" ' +
                  'text-anchor="middle" letter-spacing="4">' + text + '</text>' +
              '</svg>';
  return 'data:image/svg+xml;utf8,' + encodeURIComponent(svg);
}
```

The generated data URI is used both as the header `<img src="…">` and as the watermark source. No network request, no external file. This is what makes the demo truly standalone — you can email the single HTML file to anyone and it works.

Replacing with a real logo

If you want to use an actual logo image for a template (for a client demo, say):

1. Host the logo somewhere, or encode it as a data URI yourself
2. Edit `applyTemplate()` in the script block
3. Swap the line `$('out-logo').src = logoUri;` to point at your URL

For example:

```js
$('out-logo').src = t.logoImage || makeLogoSVG(t.logo.text, t.logo.color);
```

Then add `logoImage: "https://example.com/logo.svg"` to any template that has a real logo.

QR codes

Two separate QR slots, both optional.

Main QR (centre of the footer)

- Put anything into the QR Code input — URL, WhatsApp link (`https://wa.me/60123456789`), tel: link, email, vCard.
- Three presets:
  - **Website** — encodes the current template's `brand.websiteUrl`

- **Sign details** — encodes `Plate: … | Guest: … | Room: … | From: … | To: …` (useful for security staff who scan to cross-check against a register)
- **Clear** — empties the field

Contact QR (bottom-right, next to the contact info)

Toggle on under Advanced → Contact Info → "Show contact QR". The QR then **auto-detects** what's in the Contact Value field:

If the value looks like…	Encoded as	Effect when scanned
a phone number	`tel:<number>`	opens phone dialer
an email address	`mailto:<address>`	opens email composer
anything else	plain text	displays the text

So whoever picks up a sign with this enabled can just scan the QR to call security directly — no typing numbers from a printed page.

Deployment

The demo is a single HTML file with no build step and only two external dependencies (Google Fonts + qrcodejs CDN). You can deploy it anywhere static.

Cloudflare Pages (recommended, free)

1. Push the file to a GitHub or GitLab repository
2. Log in to Cloudflare → Workers & Pages → Create → Pages → Connect to Git
3. Pick your repo
4. Framework preset: **None**
5. Build command: *(leave blank)*
6. Build output directory: `/`
7. Deploy

Live at `https://<project>.pages.dev`.

GitHub Pages

1. Push to GitHub

2. Repo → Settings → Pages → Source: **Deploy from a branch** → `main` → `/ (root)`
3. URL appears after a minute at `https://<user>.github.io/<repo>/parking-demo.html`

Any other host

Drop the file on Netlify, Vercel, S3 + CloudFront, a plain Apache/nginx, or any other static host. No configuration needed.

Run locally

```bash
# Just open the file
open parking-demo.html          # macOS
start parking-demo.html         # Windows
xdg-open parking-demo.html      # Linux

# Or serve it
python3 -m http.server 8000
# then browse to http://localhost:8000/parking-demo.html
```

Fully offline

If you need zero-network operation, download `qrcode.min.js` next to the HTML file and change the `<script src="…">` line to a local path. Do the same with the Google Fonts if you want — though note that without the Noto fonts, CJK / Tamil / Gurmukhi scripts will fall back to the OS default, which is usually fine.

Printing tips

- **Paper:** A4, landscape. The tool is sized for A4, not Letter.
- **Margins:** the CSS sets `@page { margin: 0 }`. In the print dialog, pick **Margins → Default** or None. The 10 mm internal safe-area means nothing important gets clipped even on edge-unfriendly printers.
- **Background graphics:** must be **ON**. The dark plate box and accent colours rely on this. In Chrome/Edge, the checkbox is under **More settings → Background graphics**.
- **Colour vs B&W:** pick the theme in the sidebar *before* printing. Colour mode needs a colour printer (or PDF export). B&W mode is tuned for monochrome lasers and photocopiers.
- **Chrome / Edge** give the cleanest output. Firefox and Safari work but occasionally shift margins by 1–2 mm.

- **PDF export:** in the print dialog, pick "Save as PDF" as the destination. Or use the dedicated **Save PDF** button. Either way, you get a perfect A4-landscape PDF you can email or archive.

Data storage

The demo uses `localStorage` for three things:

Key	What it stores
`parking_demo_prefs_v1`	Last-used template, Advanced toggle state, per-section collapse state
`parking_demo_history_v1`	Last 10 printed / PDF'd signs

All per-browser, per-device. No network, no sync, no tracking, no analytics. Clear it via your browser's site-data settings or the **Clear all** button under Recent Signs.

If `localStorage` is unavailable (e.g. private-browsing modes in some browsers), every feature still works — preferences and history just won't persist across page reloads.

Browser support

Browser	Supported	Notes
Chrome / Edge	✅ Full	Best print output
Firefox	✅ Full	Minor margin drift possible
Safari	✅ Full	Test-print once before rolling out
IE 11	❌	Uses ES6 + CSS custom properties

File structure

```
.
├── parking-demo.html      The entire demo — HTML, CSS, JS, logos all inline
└── README-demo.md         This file
```

That's it. The demo is deliberately self-contained — no `assets/` folder, no build step, no package manager, no bundler. Edit, save, reload.

The only external runtime dependency is the **qrcodejs** library loaded from jsDelivr CDN (~15 KB). Everything else — fonts, icons, template logos, QR rendering — either bundles inline or loads via Google Fonts.

Differences from the production (`parking.html`) version

If you're comparing this demo to the production file:

Feature	Production	Demo
Branding	Single `CONFIG` block	`TEMPLATES` dict, picker at top
Logo	External PNG file	Generated inline SVG from initials
Templates	1 (hardcoded)	8 preconfigured, extensible
Room-badge label	Always "Room"	Template-specific ("Suite", "Table", etc.)
Reset behaviour	Resets to CONFIG defaults	Resets to current template's defaults
Recent signs	Plate + name + date	Plate + name + date + template
localStorage namespaces	`parking_sign_*_v1`	`parking_demo_*_v1` (no overlap)

Everything else — UI, sections, advanced toggle, collapsibility, languages, contact QR, signature, expired warning, past-date validation, accessibility, keyboard shortcuts — is identical.

Customisation recipes

Things you're likely to want to do. All edits are in `parking-demo.html`, inside the `<script>` block near the top.

Remove a template

Delete its entry from the `TEMPLATES` object. The grid re-renders automatically on the next page load.

Change which template loads by default

Find this line in the `DOMContentLoaded` handler:

```js
const startKey = getSavedTemplate() || 'hotel';
```

Change `'hotel'` to any template key. Note that `getSavedTemplate()` only returns a value if the user has previously clicked a template, so the fallback (`'hotel'` here) only applies on first visit.

Change the accent colour of an existing template

Edit `colors.accent` in that template's entry. B&W mode automatically overrides with black regardless, so you don't need to worry about monochrome fallback.

Change the default plate / guest for every template

Each template has its own `defaults.plate` and `defaults.guestName` — edit them individually, or write a small loop above the `applyTemplate` call at boot:

```js
Object.values(TEMPLATES).forEach(t => { t.defaults.plate = ''; t.defaults.guestName = ''; })
```

Ship a real logo instead of the generated one

See **Replacing with a real logo** above.

Add a 6th language

In the HTML, add a new `<option>` to both `<select id="in-lang-primary">` and `<select id="in-lang-secondary">`. Then in each template's `signTitles`, add a translation under the new key. Also extend `LANG_CODES` at the top of the script to map the key to an ISO `lang` attribute.

Licence

Use it, fork it, rebrand it, ship it. No attribution required.

PS: there are a few buried extras in the code. If you're the kind of dev who reads source before running things, you'll find them. ⚡